

Food Pantry - Database Schema

Version:	1.0
Date:	July 22, 2025
Revision:	
Prepared by:	OCAWEB Development Team
Description:	This document provides a structured overview of the Food Pantry application's database schema, including entity definitions, relationships, attributes, and cardinalities to support application functionality and data integrity.

Table of Contents

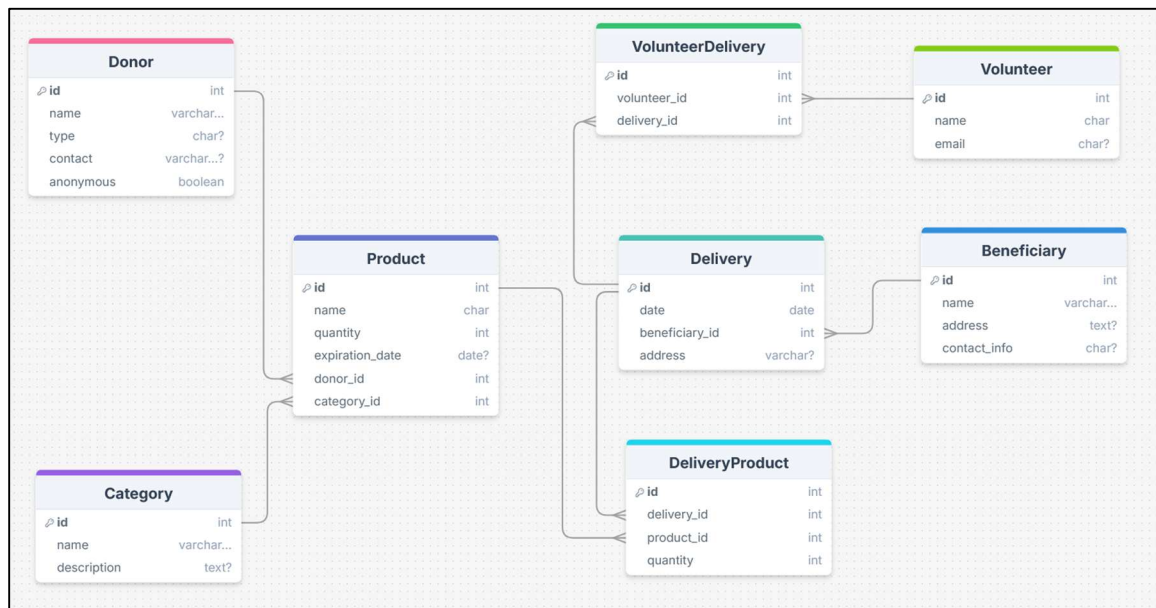
1. Introduction.....	3
2. ER Diagram.....	3
3. Entities Overview.....	4
Donor.....	4
Product.....	4
Category.....	4
Beneficiary	4
Delivery	4
DeliveryProduct.....	4
Volunteer.....	4
VolunteerDelivery.....	4
4. Relationships and Cardinality.....	5
5. Attribute Tables.....	8
Donor	8
Product.....	8
Category.....	9
Beneficiary.....	9
Delivery.....	9
DeliveryProduct	10
Volunteer.....	10
VolunteerDelivery	10
6. SQL code	11

1. Introduction

This document describes the relational database schema of the **Food Pantry** system, which supports operations such as product distribution, volunteer coordination, donor registration, and beneficiary management.

2. ER Diagram

The following image represents the Entity-Relationship Model (ERM) for the Food Pantry system, obtained after the Chen and BAKER normalization diagrams that can be seen in this summary.



Entity-Relationship Model (ERM) for the Food Pantry system.

3. Entities Overview

Donor

Stores information about the people or organizations that donate food.

Product

Represents food items available for delivery.

Category

Classifies products into categories like fruits, vegetables, etc.

Beneficiary

Stores people who receive food deliveries.

Delivery

Links beneficiaries to deliveries with date and address.

DeliveryProduct

Intermediate table linking deliveries and products (many-to-many).

Volunteer

Stores volunteers who assist with deliveries.

VolunteerDelivery

Links volunteers to deliveries (many-to-many).

4. Relationships and Cardinality

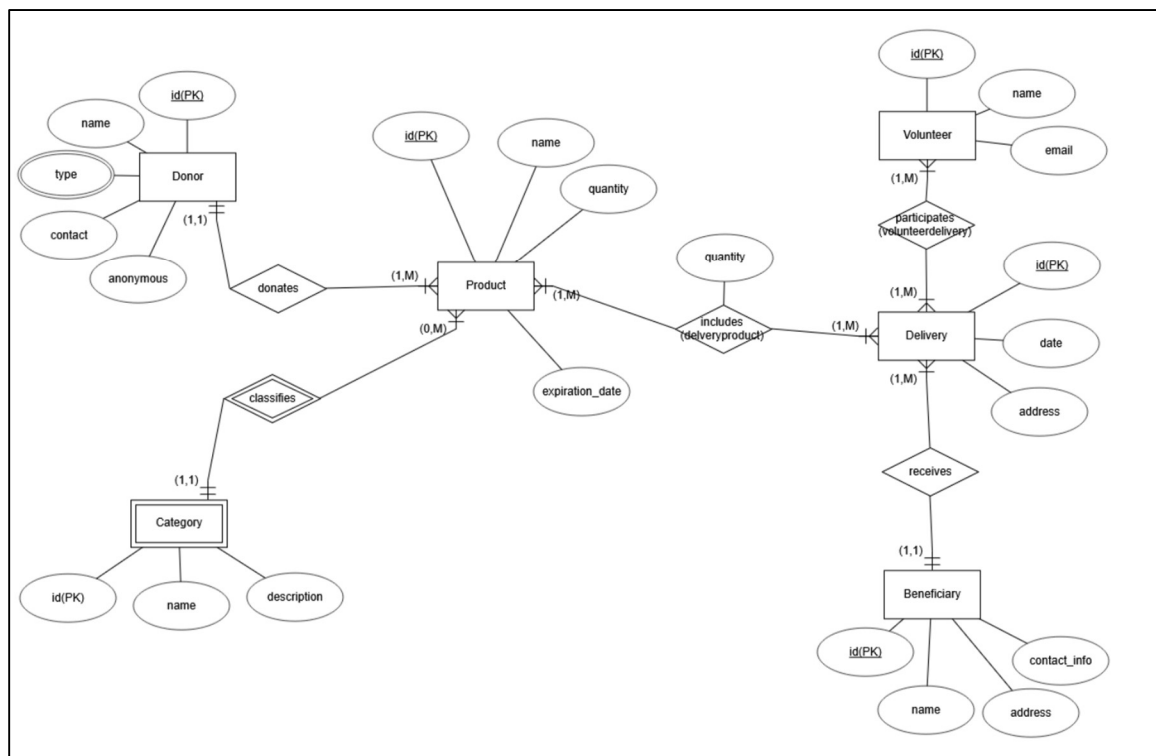
- A Donor can donate multiple Products (1:N).
- A Category can group multiple Products (1:N).
- A Product can belong to one Donor and one Category.
- A Beneficiary can receive multiple Deliveries (1:N).
- A Delivery can contain multiple Products through DeliveryProduct (N:M).
- A Volunteer can participate in multiple Deliveries through VolunteerDelivery (N:M).

Relationship	Cardinality	Description
Donor → Product	1:N	A Donor can donate multiple Products (1:N).
Category → Product	1:N	A Category can group multiple Products (1:N).
Beneficiary → Delivery	1:N	A Beneficiary can receive multiple Deliveries (1:N).
Delivery ↔ Product	N:M (via DeliveryProduct)	A Delivery can contain multiple Products through DeliveryProduct (N:M).
Delivery ↔ Volunteer	N:M (via VolunteerDelivery)	A Volunteer can participate in multiple Deliveries through VolunteerDelivery (N:M).

Extra relationships and restrictions implemented in the code logic:

Entity 1	Relationship	Entity 2	Cardinality	Description
Donor	donates	Product	1:N (one to many)	A donor can donate multiple products, but each product is donated by one donor.
Category	classifies	Product	1:N (one to many)	A category can group multiple products, but each product belongs to one category.
Product	included in	DeliveryProduct	1:N (one to many)	A product can be included in multiple deliveries via the intermediate table.
Delivery	contains	DeliveryProduct	1:N (one to many)	A delivery can contain many products.
Beneficiary	receives	Delivery	1:N (one to many)	A beneficiary can receive many deliveries, each

				delivery is assigned to one beneficiary.
Delivery	performed by	VolunteerDelivery	1:N (one to many)	A delivery can be performed by multiple volunteers (through intermediate table).
Volunteer	participates in	VolunteerDelivery	1:N (one to many)	A volunteer can participate in multiple deliveries.

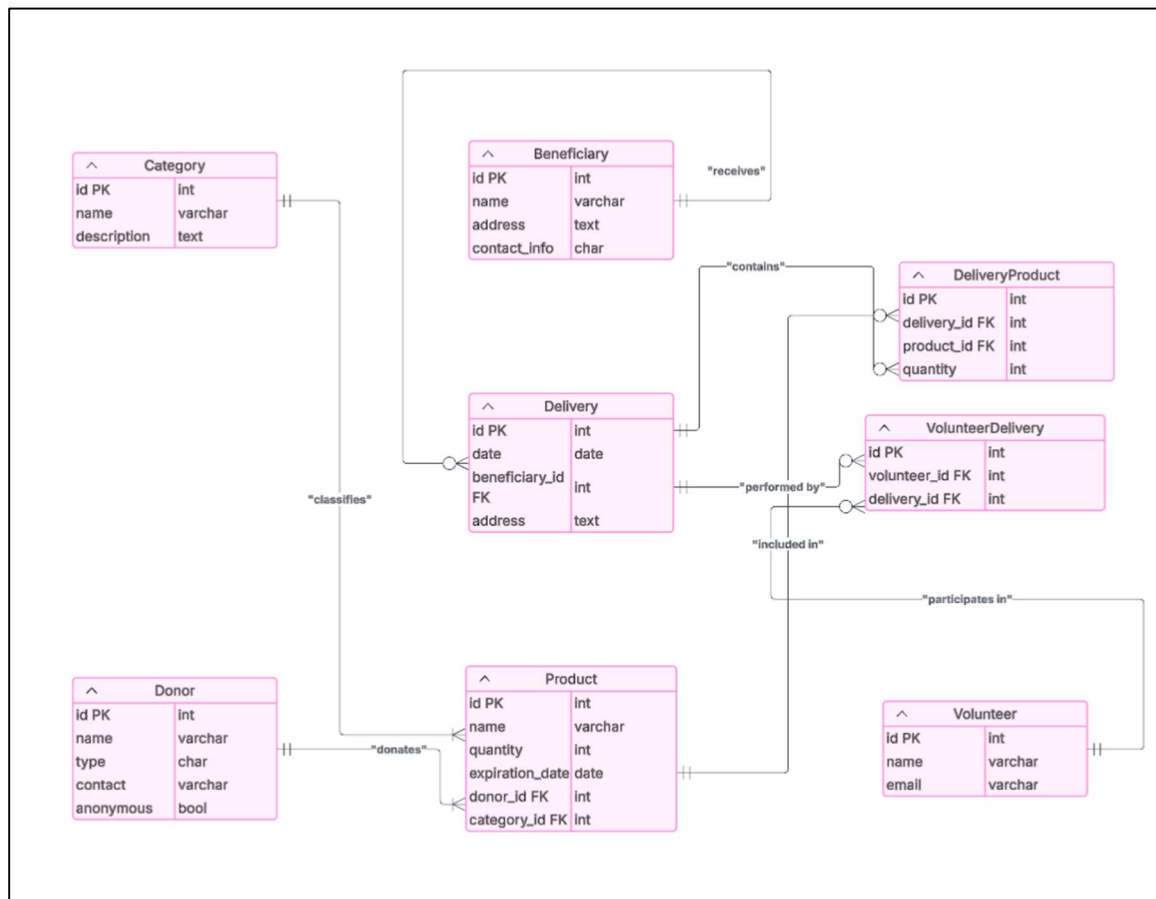


Entity-Relation diagram in Chen notation.

The relation between Product and Delivery generates the DeliveryProduct table.

The relation between Delivery and Volunteer generates the VolunteerDelivery table.

These tables, DeliveryProduct and VolunteerDelivery, can be showed in the following entire E-R diagram in BAKER notation:



Schema with cardinality in Baker normalization.

5. Attribute Tables

Donor

-- Table storing donor information (anonymous or identified)

Attribute	Type / Constraint
id	int (PK)
name	varchar(180)
type	char
contact	varchar(180)
anonymous	boolean (default=false)

*Nota DONOR_TYPES = [("individual", "Individual"), ("institución", "Institución")]

Product

-- Table storing donated products

Attribute	Type / Constraint
id	int (PK)
name	varchar(180)
quantity	int
expiration_date	date
donor_id	int (FK)
category_id	int (FK)

Category

-- Table classifying product categories
-- This table is used to categorize products for better organization and searchability
-- It allows products to be grouped by type, such as "Canned Goods", "Fresh"

Attribute	Type / Constraint
id	int (PK)
name	varchar(180)
description	text(blank=True, Null=True)

Beneficiary

-- Table storing information about product beneficiaries

Attribute	Type / Constraint
id	int (PK)
name	varchar(180)
address	text
Contact_info	char

Delivery

-- Table recording each delivery to a beneficiary

Attribute	Type / Constraint
id	int (PK)
date	date
beneficiary_id	int (FK)
address	text

DeliveryProduct

-- Table recording each delivery to a beneficiary

Attribute	Type / Constraint
id	int (PK)
delivery_id	int (FK)
product_id	int (FK)
quantity	int

Volunteer

-- Table for volunteers involved in logistics

Attribute	Type / Constraint
id	int (PK)
name	varchar(180)
email	Varchar(180)

VolunteerDelivery

-- Table for volunteers involved in logistics

Attribute	Type / Constraint
id	int (PK)
volunteer_id	int (FK)
delivery_id	int (FK)

6. SQL code

Here, it provides the SQL code, but the code is implemented in Django models.py of every api (donors_api, products_api, volunteers_api, deliveries_api,...).

```
-- Table storing donor information (anonymous or identified)

CREATE TABLE "Donor"(

    "id" INTEGER NOT NULL,

    "name" VARCHAR(180) NOT NULL,

    "type" CHAR(255) NULL,

    "contact" VARCHAR(180) NULL,

    "anonymous" BOOLEAN NOT NULL

);

ALTER TABLE

    "Donor" ADD PRIMARY KEY("id");

-- Table classifying product categories

-- This table is used to categorize products for better organization and
searchability

-- It allows products to be grouped by type, such as "Canned Goods", "Fresh

CREATE TABLE "Category"(

    "id" INTEGER NOT NULL,

    "name" VARCHAR(180) NOT NULL,

    "description" TEXT NULL

);

ALTER TABLE

    "Category" ADD PRIMARY KEY("id");

-- Table storing donated products
```

```
CREATE TABLE "Product"(  
    "id" INTEGER NOT NULL,  
    "name" CHAR(255) NOT NULL,  
    "quantity" INTEGER NOT NULL,  
    "expiration_date" DATE NULL,  
    "donor_id" INTEGER NOT NULL,  
    "category_id" INTEGER NOT NULL  
);  
  
ALTER TABLE  
    "Product" ADD PRIMARY KEY("id");  
  
-- Table storing information about product beneficiaries  
  
CREATE TABLE "Beneficiary"(  
    "id" INTEGER NOT NULL,  
    "name" VARCHAR(180) NOT NULL,  
    "address" TEXT NULL,  
    "contact_info" CHAR(255) NULL  
);  
  
ALTER TABLE  
    "Beneficiary" ADD PRIMARY KEY("id");  
  
-- Table recording each delivery to a beneficiary  
  
CREATE TABLE "Delivery"(  
    "id" INTEGER NOT NULL,  
    "date" DATE NOT NULL,  
    "beneficiary_id" INTEGER NOT NULL,
```

```
        "address" CHAR(255) NULL
    );

ALTER TABLE

    "Delivery" ADD PRIMARY KEY("id");

-- Table recording each delivery to a beneficiary
CREATE TABLE "DeliveryProduct"(
    "id" INTEGER NOT NULL,
    "delivery_id" INTEGER NOT NULL,
    "product_id" INTEGER NOT NULL,
    "quantity" INTEGER NOT NULL
);

ALTER TABLE

    "DeliveryProduct" ADD PRIMARY KEY("id");

-- Table for volunteers involved in logistics
CREATE TABLE "Volunteer"(
    "id" INTEGER NOT NULL,
    "name" CHAR(100) NOT NULL,
    "email" VARCHAR(255) NULL
);

ALTER TABLE

    "Volunteer" ADD PRIMARY KEY("id");

-- Table for volunteers involved in logistics
CREATE TABLE "VolunteerDelivery"(
```

```
"id" INTEGER NOT NULL,  
"volunteer_id" INTEGER NOT NULL,  
"delivery_id" INTEGER NOT NULL  
);  
  
ALTER TABLE  
    "VolunteerDelivery" ADD PRIMARY KEY("id");  
  
ALTER TABLE  
    "DeliveryProduct" ADD CONSTRAINT "deliveryproduct_product_id_foreign"  
FOREIGN KEY("product_id") REFERENCES "Product"("id");  
  
ALTER TABLE  
    "VolunteerDelivery" ADD CONSTRAINT  
"volunteerdelivery_volunteer_id_foreign" FOREIGN KEY("volunteer_id")  
REFERENCES "Volunteer"("id");  
  
ALTER TABLE  
    "Delivery" ADD CONSTRAINT "delivery_beneficiary_id_foreign" FOREIGN  
KEY("beneficiary_id") REFERENCES "Beneficiary"("id");  
  
ALTER TABLE  
    "VolunteerDelivery" ADD CONSTRAINT "volunteerdelivery_delivery_id_foreign"  
FOREIGN KEY("delivery_id") REFERENCES "Delivery"("id");  
  
ALTER TABLE  
    "DeliveryProduct" ADD CONSTRAINT "deliveryproduct_delivery_id_foreign"  
FOREIGN KEY("delivery_id") REFERENCES "Delivery"("id");  
  
ALTER TABLE  
    "Product" ADD CONSTRAINT "product_donor_id_foreign" FOREIGN  
KEY("donor_id") REFERENCES "Donor"("id");  
  
ALTER TABLE  
    "Product" ADD CONSTRAINT "product_category_id_foreign" FOREIGN  
KEY("category_id") REFERENCES "Category"("id");
```